

Engineering Learnings

What building a DRC-repair agent taught us

Companion to the **ASU problem** deck — the research and experiments behind the design.

Built in one focused day; every claim is measured and reproducible.

Harikrishnan KC · Team **Chip Convergence** · greatharikrishnan@gmail.com

1 · Version-exactness is a first-class requirement

The evaluator is a **string-equality gate**: `klayout -v` must read exactly `0.30.1`, or DRC is refused. macOS Homebrew gives 0.30.9 (and Gatekeeper flags the unsigned app).

We initially fought the macOS install; the right move was to **stop changing the host and go Docker** — an amd64 image with the pinned `.deb`, run under emulation on Apple Silicon. The payoff wasn't just "it runs": our container DRC matches the reference report on **11/14 rules exactly**, proving our environment *is* the scoring environment.

Lesson: for a benchmark scored in a pinned environment, reproduce that environment exactly before trusting a single local number. Docker over host, every time.

2 · Measure with the scorer's own code, or don't trust the number

Our first instinct was to parse DRC reports and count violations ourselves. Instead we **import the official `evaluate_repair.py` functions** — the same render, DRC, count, and connectivity routines the organizers score with.

Why it mattered immediately: feeding the *unmodified* script back scored `final_violation_rate` = 1.29, not 1.0. A per-rule diff (11/14 exact; 3 grid rules inflated by clean 2x/4x factors) localized the gap to a script-rebuild multiplicity artifact — a diagnosis we could only trust because our counter *was* their counter.

Lesson: an inner-loop metric that isn't byte-identical to the scoring metric will optimize the wrong thing. Reuse the scorer.

3 · Read the rule deck, not the error message

We spent a first pass reverse-engineering fixes from the DRC report's human descriptions ("V2 must be the same width as M3..."). Every fix regressed.

The descriptions are lossy. The **actual** `asap7.lydrc` **rule** is precise:

`V2.M3.AUX.2` is satisfied iff the via is *inside* M3 **and has ≥ 2 edges coincident with M3's edges** — and it is coupled to `V2.M2.EN.1` (5 nm enclosure) and `V2.AUX.1` (containment). Once we read the deck, the fix became *derivable* instead of guessable.

Lesson: the authoritative spec is the checker's source, not its output. Read it.

4 · Research reframes the whole problem

One web search was not enough for a specialized domain. Digging into **LAD and ISPD** surfaced the reframe:

- **DRC-Coder (ISPD'25, NVIDIA)** — vision + LLM interpret DRC rules at F1=1.0. Validates the untapped **screenshot** lever (Gemini is multimodal).
- **Wide-metal via enclosure** (PDK/patents) — a *wide* metal needs a *wide* via, not a min-via. That is exactly our 181 via-width-match violations: V2 (min) sitting in a wide M3.
- **MDPI 2025** — standard-cell DRC repair as **simulated annealing over a conflict graph**: the formal model for what we'd found empirically.

Lesson: the literature had already named our problem. An hour of reading changed the plan more than a day of coding.

5 · The coupling is the problem (measured, not asserted)

Every single-layer fix worked on its target rule and broke a neighbour — we have the table:

fix	fixes	breaks	net
grid-snap (whole layer)	grid	via enclosure	315→510
grow via → metal width	AUX.2	lower-metal enclosure	315→387
shrink metal → via width	AUX.2	upper-via enclosure	315→339
coordinated via + metal patch	AUX.2 + enclosure	neighbour spacing	315→379

The coordinated fixer (derived from the exact rule) was the first to **hold enclosure** — proving the recipe was right — but a wide via forces a wide metal that crowds neighbours. The violation is conserved; it just moves through the via stack.

Lesson: in coupled-constraint repair, "fixed the target rule" is not progress unless the net count drops. Measure the net, always.

6 · Safety-by-construction beats cleverness

We never needed a "don't break connectivity" heuristic. Two structural choices made regressions *impossible to ship*:

1. **Repairs are appended passes on the original script** — original shapes untouched, so the statically-checked connectivity is preserved by construction.
2. **Keep-best over the eligible baseline** — every candidate is verified with the scorer's own code; anything that regresses or breaks eligibility is discarded.

Result: across all 5 blocks and every experimental pass (including a model that over-corrected 315→13,823), the agent always emitted an eligible, connectivity-preserved script.

Lesson (shared with our NVIDIA/NXP work): make the bad outcome unrepresentable, then explore aggressively. The verification spine is what lets you take risks safely.

Harikrishnan KC · Chip Convergence · greatharikrishnan@gmail.com